Lecture Slides: Database

• Slide 1 - Database Fundamentals**

• Subtitle:* Survey of theoretical foundations, architectural components, and contemporary trends in modern database systems.

• Core data-management concepts

• Relational vs. non-relational paradigms

• Transactional integrity, scalability, and emerging hybrid workloads

•

• Slide 2 - Learning Objectives**

By the end of the lecture students will be able to:

1. **Define** and **differentiate** a *database*, a *DBMS*, and a *data model* using precise terminology.

2. **Explain** the ACID properties and **illustrate** how concurrency-control mechanisms enforce them.

3. **Compare** relational, NoSQL, and NewSQL architectures in terms of schema flexibility, scalability, and consistency guarantees.

4. **Analyse** a workload and **recommend** appropriate indexing, normalization, or denormalization strategies.

•

• Slide 3 - Introduction & Context**

• **Database** = organized collection of inter-related data stored electronically for efficient access, management, and update.

• **DBMS** abstracts physical storage and offers declarative interfaces (e.g., SQL) for definition, manipulation, and control.

• **Data models** are logical blueprints; the relational model dominates enterprises, while NoSQL models serve high-velocity, schema-agnostic workloads.

• **Transaction processing** guarantees reliability through ACID properties-essential for mission-critical OLTP applications.

• **2024 market snapshot:** ~70 % of enterprise deployments use relational DBMSs; NoSQL and NewSQL are rapidly gaining ground in cloud-native and real-time analytics contexts.

•

• Slide 4 - Core Concept 1: Database Management Systems (DBMS)**

• **Definition:** Software that implements the full data lifecycle-creation, retrieval, update, deletion-while enforcing integrity constraints.

• **Concurrency control:** Locking, timestamp ordering, or **Multiversion Concurrency Control (MVCC)** preserve isolation.

• **Security services:** Authentication, role-based authorization, encryption (at rest & in transit), and audit logging.

• **Metadata management:** A data dictionary/catalog records schema definitions, data types, and system objects for self-description.

• **Backup & recovery:** APIs support full, incremental, and point-in-time restores, mitigating data-loss risk.

• **Modern DBMSs expose** programmable interfaces (REST, gRPC, JDBC/ODBC) and built-in tools for automated failover, scaling, and observability.

•

• Slide 5 - Core Concept 2: Data Models & the Relational Paradigm**

• **Data model:** Abstracts structure, relationships, and constraints; guides schema design and query formulation.

• **Relational model:** Represents data as *tuples* grouped into *relations* (tables); grounded in first-order predicate logic.

• **SQL sub-languages:**

• DDL - schema definition

• DML - data manipulation

• DCL - data control (permissions)

• TCL - transaction control (COMMIT/ROLLBACK)

• **Normalization:** Decomposes relations into higher normal forms (1NF-5NF, BCNF) to eliminate redundancy and update anomalies.

• **Denormalization:** Re-introduces controlled redundancy to boost read performance in analytical or distributed environments.

•

• Slide 6 - Core Concept 3: Transaction Management & Concurrency Control**

• **Transaction:** Logical unit of work that must **commit** entirely or **rollback** entirely, preserving consistency.

• **Atomicity:** Achieved via *write-ahead logging* (WAL) - intent recorded before page modification.

• **Consistency:** Enforced by integrity constraints (PK, FK, CHECK) validated at commit time.

• **Isolation levels:**

• READ COMMITTED - prevents dirty reads

• REPEATABLE READ - prevents non-repeatable reads

• SERIALIZABLE - eliminates phantom rows (full isolation)

• **Durability:** Persisted through checkpointing, replication, and storage-level guarantees.

• **Pessimistic vs. optimistic:** Lock-based (pessimistic) vs. MVCC (optimistic) illustrate different philosophies for achieving isolation.

•

• Slide 7 - Deep Dive: Query Optimization & Indexing**

• **Query optimizer:** Transforms declarative SQL into an execution plan that minimizes estimated cost using statistics & heuristics.

• **Cost-based optimizer:** Evaluates alternative join orders, access paths, and operator implementations via a mathematical cost model.

• **Rule-based optimizer:** Applies predefined transformations (e.g., predicate push-down) without runtime statistics.

• **Index structures:**

• **B-tree** - point lookups & range scans

• **Hash index** - exact-match queries

• **Bitmap index** - high-cardinality, analytical aggregations

• **Index trade-offs:** Selectivity, write overhead, storage consumption, and query latency must be balanced.

•

• Slide 8 - Real-World Applications & Examples**

| Domain | Typical Workload | Representative DBMS | Key Architectural Feature |

|--------|------------------|---------------------|---------------------------|

| **Finance (OLTP)** | High-throughput, strict ACID | Oracle, PostgreSQL | Row-store, MVCC, strong consistency |

| **E-commerce (HTAP)** | Mixed OLTP/OLAP, real-time personalization | CockroachDB, TiDB | Distributed SQL, global consistency, built-in HTAP |

| **Analytics (OLAP)** | Complex aggregations, columnar scans | Amazon Redshift, Snowflake | Columnar storage, materialized views |

| **Content Management (NoSQL)** | Schema-flexible, rapid iteration | MongoDB, Couchbase | Document model, horizontal scaling |

| **Real-time Recommendation** | Sub-millisecond latency, in-memory | Redis, SAP HANA | In-memory, data-grid, high throughput |

•

• Slide 9 - Critical Thinking & Analysis**

1. **CAP theorem trade-offs** - Design a globally distributed social-media platform: which guarantees (Consistency, Availability, Partition tolerance) are prioritized and why?

2. **Normalization vs. performance** - When does excessive normalization hurt read-heavy analytical workloads? Propose a denormalization strategy.

3. **Security in multi-tenant clouds** - Discuss encryption-at-rest, encryption-in-transit, and audit logging for PII.

4. **Scaling economics** - Compare vertical scaling (scale-up) vs. horizontal scaling (scale-out) for relational vs. NoSQL systems (cost, operational complexity).

5. **NoSQL for financial ledgers** - Critique the suitability of a pure key-value store for a ledger that requires strict transactional guarantees.

•

• Slide 10 - Summary & Closing Discussion**

• **DBMS** provides a unified framework for definition, manipulation, security, and recovery.

• **Relational, NoSQL, NewSQL** each address distinct trade-offs among consistency, scalability, and schema flexibility.

• **Transactions, query optimization, and indexing** are the core mechanisms that reconcile performance with correctness.

• Discussion Prompt**

• "A rapidly growing e-commerce platform must support high-volume transactions *and* real-time personalization. Which combination of database technologies would you propose and why?"*

• Further Reading**

• Recent peer-reviewed survey: *Hybrid Transactional/Analytical Processing (HTAP) Architectures* (2024).

•